SDM Array Model
Equations & Calculations

Reference for `sdm_array_model.py`

Contents

1 Overview **2**
- 1.1 Physical Constants . . . 2
- 1.2 Datasheet & Array Structures . . . 2

2 DC Derating **2**

3 Thermal Model (Faiman) **3**

4 SDM Primitives **3**
- 4.1 Thermal Voltage (module) . . . 3
- 4.2 Single-Diode IV Equation . . . 3
- 4.3 Derivative $\mathrm{d}I/\mathrm{d}V$ . . . 3

5 Fit SDM from STC **4**

6 Voc and MPP Solvers **4**
- 6.1 Open-Circuit Voltage (bisection) . . . 4
- 6.2 MPP (golden-section on $P(V) = V\,I(V)$) . . . 4

7 Real-Time Power Estimation **4**
- 7.1 Cell temperature . . . 4
- 7.2 Thermal voltage at operating T . . . 5
- 7.3 Saturation current at T . . . 5
- 7.4 Photocurrent at (G, T) . . . 5
- 7.5 Module MPP . . . 5
- 7.6 String and array scaling . . . 5
- 7.7 DC derating . . . 5

8 Degradation (installation age) **5**

9 Flowcharts **6**
- 9.1 Overall calculation flow . . . 6
- 9.2 Parameter resolution (fitted SDM params) . . . 6
- 9.3 IV current (Newton) and MPP . . . 7

10 Version history (upgradations) **7**

1 Overview

The Single Diode Model (SDM) Array implementation:

1. Fits SDM parameters ($I_{\text{ph}}, I_0, R_s, R_{\text{sh}}, n$) from module datasheet (STC) once.
2. Reuses those fitted parameters for all subsequent power calculations.
3. Uses IV curve solving (Newton) and MPP finding (golden-section / bisection).

1.1 Physical Constants

$$k = 1.380649 \times 10^{-23}\ \text{J/K} \qquad q = 1.602176634 \times 10^{-19}\ \text{C} \tag{1}$$

1.2 Datasheet & Array Structures

Module datasheet (STC):

- I_{sc} — short-circuit current (A), V_{oc} — open-circuit voltage (V)
- $I_{\text{mp}}, V_{\text{mp}}$ — current and voltage at MPP (A, V)
- N_s — series cells per module
- E_g — bandgap (eV), Si ≈ 1.12 eV
- α_I — I_{sc} temperature coefficient (A/°C)
- β_V — V_{oc} temperature coefficient (fraction per °C)

Array configuration:

- N_{ser} — modules in series per string
- N_{par} — strings in parallel
- NOCT — Nominal Operating Cell Temperature (°C)

2 DC Derating

Loss fractions (each in $[0, 1]$): soiling, mismatch, dc_ohmic, degradation, availability, others.

$$\text{total_loss} = 1 - \prod_j (1 - L_j) \tag{2}$$

$$\boxed{\eta_{\text{dc}} = 1 - \text{total_loss}} \tag{3}$$

So $P_{\text{dc}} = P_{\text{dc,raw}} \cdot \eta_{\text{dc}}$.

3 Thermal Model (Faiman)

Cell temperature from plane-of-array irradiance G, ambient temperature T_{amb} (°C), and wind speed v_{wind} (m/s):

$$U_{\text{sum}} = \frac{800}{\max(\text{NOCT} - 20,\ 10^{-6})} \quad (\text{W/m}^2\text{-K}) \tag{4}$$

$$U_0 = 0.8\,U_{\text{sum}}, \qquad U_1 = 0.2\,U_{\text{sum}} \tag{5}$$

$$\boxed{T_{\text{cell}} = T_{\text{amb}} + \frac{G}{U_0 + U_1 \cdot \max(v_{\text{wind}}, 0)}} \tag{6}$$

4 SDM Primitives

4.1 Thermal Voltage (module)

$$\boxed{V_t = \frac{k\,T}{q}\,N_s} \tag{7}$$

with T in Kelvin.

4.2 Single-Diode IV Equation

The output current I at voltage V is given implicitly by:

$$\boxed{I = I_{\text{ph}} - I_0\left(\exp\frac{V + IR_s}{n\,V_t} - 1\right) - \frac{V + IR_s}{R_{\text{sh}}}} \tag{8}$$

Newton iteration: define

$$e_a = \frac{V + IR_s}{n\,V_t}, \quad e_a \in [-60,\,60] \tag{9}$$

$$e = \exp(e_a) \tag{10}$$

Residual and derivative with respect to I:

$$f(I) = I - I_{\text{ph}} + I_0(e - 1) + \frac{V + IR_s}{R_{\text{sh}}} \tag{11}$$

$$\frac{\partial f}{\partial I} = 1 + I_0\,e\,\frac{R_s}{n\,V_t} + \frac{R_s}{R_{\text{sh}}} \tag{12}$$

Update:

$$\Delta I = \frac{f}{\max(\partial f/\partial I,\ 10^{-16})}, \qquad I_{\text{new}} = I - \text{clip}(\Delta I,\ -2,\ 2) \tag{13}$$

Iterate until $|I_{\text{new}} - I| < 10^{-10}$; then $I \leftarrow \max(I_{\text{new}},\ 0)$.

4.3 Derivative $\mathrm{d}I/\mathrm{d}V$

$$A = \frac{I_0\,e}{n\,V_t} + \frac{1}{R_{\text{sh}}}, \qquad B = 1 + R_s\,A \tag{14}$$

$$\boxed{\frac{\mathrm{d}I}{\mathrm{d}V} = -\frac{A}{\max(B,\ 10^{-16})}} \tag{15}$$

At MPP: $I_{\text{mp}} + V_{\text{mp}}\,\frac{\mathrm{d}I}{\mathrm{d}V}\big|_{V_{\text{mp}}} = 0$.

5 Fit SDM from STC

Given module datasheet at $T_{\text{cell}} = 25\text{C}$: fit $\theta = (I_{\text{ph}},\ \log_{10} I_0,\ R_s,\ R_{\text{sh}},\ n)$.

Bounds:

$$I_{\text{ph}} \in [0.98\, I_{\text{sc}},\ 1.02\, I_{\text{sc}}] \tag{16}$$

$$\log_{10} I_0 \in [-40,\ -2] \tag{17}$$

$$R_s \in [10^{-4},\ 2], \quad R_{\text{sh}} \in [10,\ 5000], \quad n \in [1,\ 2.5] \tag{18}$$

Residuals (to be driven to zero by least-squares):

$$r_1 = I(V{=}0) - I_{\text{sc}} \tag{19}$$

$$r_2 = I(V{=}V_{\text{oc}}) - 0 \tag{20}$$

$$r_3 = I(V{=}V_{\text{mp}}) - I_{\text{mp}} \tag{21}$$

$$r_4 = I_{\text{mp}} + V_{\text{mp}} \left.\frac{\mathrm{d}I}{\mathrm{d}V}\right|_{V_{\text{mp}}} \quad \text{(MPP slope)} \tag{22}$$

$$r_5 = I_{\text{ph}} - I_{\text{sc}} \quad \text{(soft tie)} \tag{23}$$

Multiple seeds over $R_s \in \{0.1,\ 0.3,\ 0.6\}$, $R_{\text{sh}} \in \{200,\ 500,\ 1000\}$, $n \in \{1.15,\ 1.3,\ 1.5\}$; initial I_0 guess:

$$I_0^{\text{guess}} = \frac{I_{\text{sc}}}{\exp(V_{\text{oc}}/(n\, V_t)) - 1 + 10^{-30}} \tag{24}$$

Best solution (lowest cost) is kept; returns $I_{\text{ph}}, I_0, R_s, R_{\text{sh}}, n, V_{t,\text{ref}}, T_{\text{ref}}$.

6 Voc and MPP Solvers

6.1 Open-Circuit Voltage (bisection)

Initial guess (ideal diode):

$$V_{\text{oc}}^{\text{guess}} = n\, V_t\, \ln\left(1 + \frac{I_{\text{ph}}}{\max(I_0,\ 10^{-30})}\right) \tag{25}$$

Bracket: $[0,\ V_{\text{oc}}^{\text{guess}}]$; expand upper bound until $I(V_{\text{hi}}) \le 0$. Bisect until $|\text{hi} - \text{lo}| < 10^{-7}$; $V_{\text{oc}} = (\text{lo} + \text{hi})/2$.

6.2 MPP (golden-section on $P(V) = V\, I(V)$)

$$\varphi = \frac{\sqrt{5} - 1}{2}, \qquad \text{bracket: } [a, b] = [0,\ 0.999\, V_{\text{oc}}] \tag{26}$$

Golden-section search on $P(V)$ until $|b - a| < 10^{-6}$; then

$$V_{\text{mp}} = \frac{a + b}{2}, \qquad I_{\text{mp}} = I(V_{\text{mp}}), \qquad P_{\text{mp}} = V_{\text{mp}}\, I_{\text{mp}} \tag{27}$$

7 Real-Time Power Estimation

Inputs: G_{poa}, T_{amb}, v_{wind}, module datasheet, array config, derates, fitted params.

7.1 Cell temperature

$$T_{\text{cell}} = \text{Faiman}(G_{\text{poa}}, T_{\text{amb}}, v_{\text{wind}}, \text{NOCT}) \tag{28}$$

$T = T_{\text{cell}} + 273.15$, $T_{\text{ref}} = T_{\text{ref},C} + 273.15$.

7.2 Thermal voltage at operating T

$$V_t = \frac{k\,T}{q}\,N_s \tag{29}$$

7.3 Saturation current at T

$$\boxed{I_0(T) = I_{0,\mathrm{ref}}\left(\frac{T}{T_{\mathrm{ref}}}\right)^3 \exp\left(\frac{q\,E_g}{n\,k}\left(\frac{1}{T_{\mathrm{ref}}} - \frac{1}{T}\right)\right)} \tag{30}$$

7.4 Photocurrent at (G, T)

$$\boxed{I_{\mathrm{ph}} = (I_{\mathrm{sc}} + \alpha_I\,(T_{\mathrm{cell}} - 25))\,\frac{G_{\mathrm{poa}}}{1000}} \tag{31}$$

7.5 Module MPP

$$(V_{\mathrm{mp,mod}},\ I_{\mathrm{mp,mod}},\ P_{\mathrm{mp,mod}}) = \mathrm{mpp_golden}(I_{\mathrm{ph}}, I_0, R_s, R_{\mathrm{sh}}, n, V_t) \tag{32}$$

7.6 String and array scaling

$$V_{\mathrm{mp,str}} = N_{\mathrm{ser}}\,V_{\mathrm{mp,mod}}, \quad I_{\mathrm{mp,str}} = I_{\mathrm{mp,mod}}, \quad P_{\mathrm{mp,str}} = V_{\mathrm{mp,str}}\,I_{\mathrm{mp,str}} \tag{33}$$

$$V_{\mathrm{mp,arr}} = V_{\mathrm{mp,str}}, \quad I_{\mathrm{mp,arr}} = N_{\mathrm{par}}\,I_{\mathrm{mp,str}} \tag{34}$$

$$P_{\mathrm{dc,raw}} = V_{\mathrm{mp,arr}}\,I_{\mathrm{mp,arr}} \tag{35}$$

7.7 DC derating

$$\boxed{P_{\mathrm{dc}} = P_{\mathrm{dc,raw}}\,\eta_{\mathrm{dc}}} \tag{36}$$

8 Degradation (installation age)

$$\mathrm{age_years} = \frac{(\mathrm{today} - \mathrm{installation_date}).\mathrm{days}}{365.25} \tag{37}$$

If `measured_degradation_rate` is set:

$$\mathrm{total_degradation_pct} = \mathrm{measured_degradation_rate} \times \mathrm{age_years} \tag{38}$$

Else (warranty-based):

$$\mathrm{total_degradation_pct} = \begin{cases} \mathrm{estimated_degradation_year1} & \text{if age_years} \le 1 \\ \mathrm{year1_deg} + (\mathrm{age_years} - 1)\,\mathrm{annual_deg} & \text{otherwise} \end{cases} \tag{39}$$

Degradation is applied as one of the loss terms in the derate factor η_{dc}.

9 Where MPP/Vmpp time is spent (code locations)

The "MPP / Vmpp calculation" timing (`estimate_power_vmpp_ms`) is the time for the whole `estimate_power()` call. Almost all of that time is inside `mpp_golden()`, which finds the maximum power point by (1) finding V_{oc} with bisection and (2) maximizing $P(V) = V\,I(V)$ with golden-section search. Both steps call `iv_current()` many times, and each `iv_current()` can run up to 60 Newton iterations. That is why this step dominates.

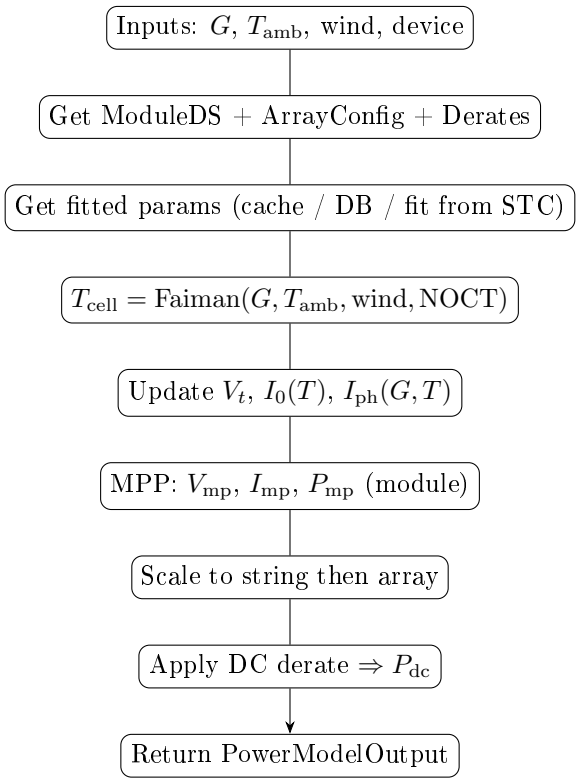
9.1 Code locations (file: `sdm_array_model.py`)

- **Timing wrapped around:** `calculate_expected_power()` calls `estimate_power()` and records the elapsed time as `estimate_power_vmpp_ms` (lines 698–708).
- **Entry:** `estimate_power()` (lines 245–283). Steps 1–3 (Faiman, V_t, I_0, I_{ph}) are cheap. Step 4 is the expensive one.
- **Expensive step:** `mpp_golden(Iph, I0, Rs, Rsh, n, Vt)` (line 263) → defined at lines 205–239.
- **Inside `mpp_golden`:**
  - `voc_bisect(...)` (line 212) → defined at lines 174–203. Up to 40 bracket expansions + 80 bisection iterations; each iteration calls `I_of_V(V)` ≡ `iv_current(V, ...)`.
  - Golden-section loop: 100 iterations (lines 219–232); each iteration evaluates `P_of_V` once or twice, and `P_of_V(V)` calls `I_of_V(V)` ≡ `iv_current(V, ...)`.
- **Core cost:** `iv_current(V, Iph, I0, Rs, Rsh, n, Vt)` (lines 91–106). For each voltage V, Newton iteration runs up to 60 times (line 94) until $|I_{\text{new}} - I| < 10^{-10}$. Each iteration uses $\exp(\cdot)$, so the total number of exponentials per MPP is roughly (bisection + golden) × Newton ≈ (80 + 200) × avg Newton.

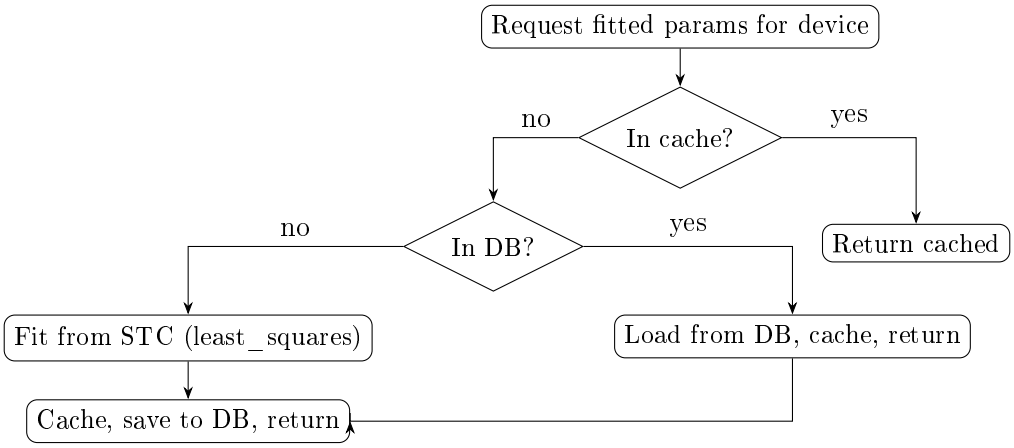
So the 211 ms (or similar) is almost entirely: `mpp_golden` → `voc_bisect` (many `iv_current`) + golden-section (many `iv_current`), with each `iv_current` doing a Newton solve.

10 Flowcharts

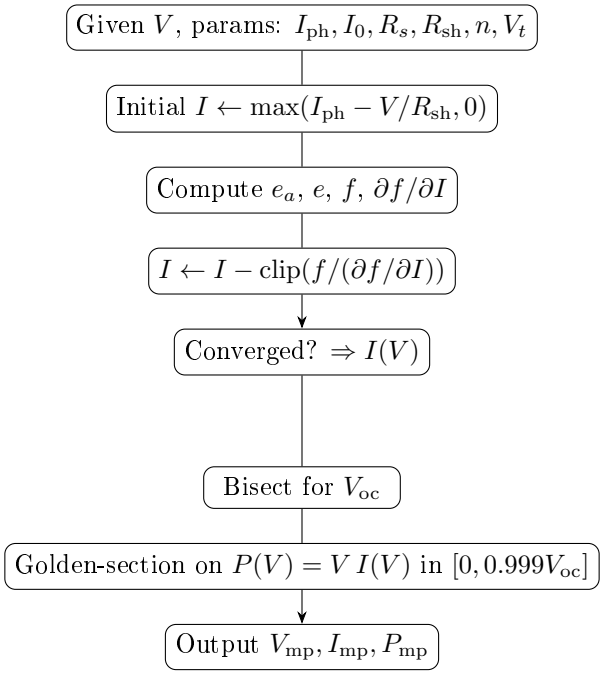
10.1 Overall calculation flow



10.2 Parameter resolution (fitted SDM params)

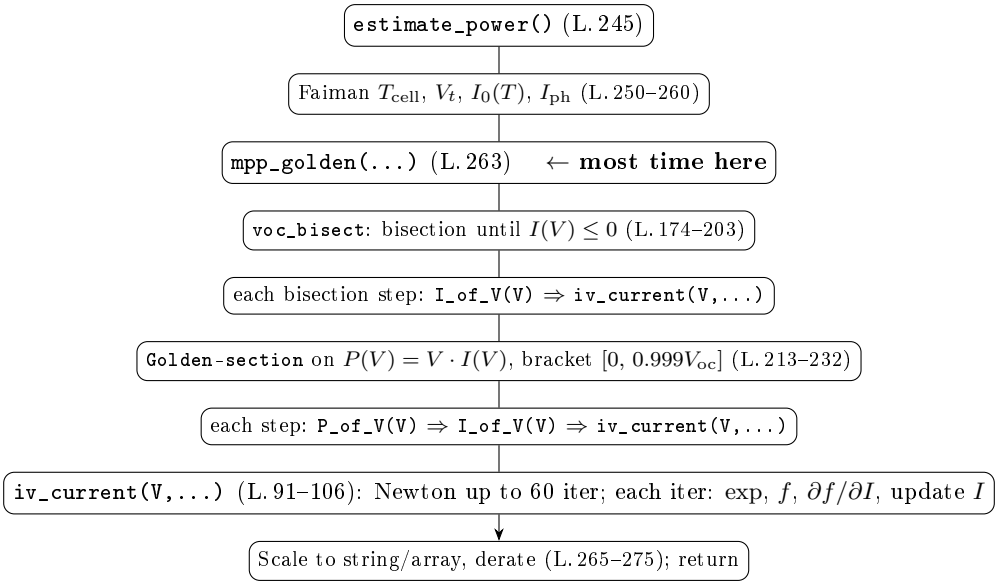


10.3 IV current (Newton) and MPP



10.4 MPP / Vmpp calculation flow (where the time goes)

The following flowchart shows exactly how the MPP step runs and why it costs the most. All timed work under "estimate_power_vmpp_ms" is inside `estimate_power()`; the dominant part is `mpp_golden`, which uses `iv_current` (Newton) many times.



Rough count per single MPP call: voc_bisect $\sim$ 80–120 evaluations of $I(V)$, golden-section $\sim$ 200 evaluations of $P(V)$ (each $= V \cdot I(V)$). So $\sim$ 280–320 calls to `iv_current`, each with up to 60 Newton iterations. That explains the high "MPP / Vmpp calculation" time (e.g. 211 ms).

11 Version history (upgradations)

Use the table below to record changes when you upgrade the model.

Date	Section / Equation	Change
(initial)	—	Document created from `sdm_array_model.py`.